# Python Foundations
# Practical Practice Questions

*Practice Set • 5 Questions • Suggested Time: 90 minutes*

## Instructions

- You have 90 minutes to complete all 5 questions. Suggested pacing is listed with each question.
- Create a new folder for this exercise and open it in VS Code. Save each solution in a clearly named Python file.
- Focus on producing a working solution before trying to make it elegant. If you get stuck, write down the exact concept you need to review.
- Do not use external libraries. Follow each question's constraints and avoid shortcuts that the question specifically prohibits.
- Test your code with the examples provided and at least one additional case of your own.

# QUESTION 1 — LIST ANALYSER

**Suggested time: 8 minutes**

## Problem

Given a list of numbers, find the smallest number, largest number, total, and average without using built-in shortcuts for these calculations.

Use this starting list: numbers = [12, 5, 27, 8, 14].

## Examples

**Input list: [12, 5, 27, 8, 14]**

```
Smallest: 5
Largest: 27
Total: 66
Average: 13.2
```

## Constraints

- Use loops and variables to calculate the results.
- Do not use min(), max(), or sum().
- You may use len() to calculate the average.
- Assume the list contains at least one number.

## What This Tests

Lists, iteration, accumulators, comparisons, and arithmetic.

Notes / concepts to review: *(Remember to write your weakness it if you have one)*

# QUESTION 2 — LINEAR SEARCH

**Suggested time: 8 minutes**

## Problem

Given a list and a target value, search through the list one item at a time. Print the index where the target first appears, or a message saying it was not found.

Use this starting list: values = [14, 7, 22, 9, 7, 31].

## Examples

**Target: 22**

```
Found at index 2
```

**Target: 18**

```
Target not found
```

## Constraints

- Use a loop; do not use the list's index() method.
- Stop searching once the first match is found.
- Do not import libraries.

## What This Tests

Lists, loops, comparisons, indexing, and control flow.

Notes / concepts to review: *(Remember to write your weakness it if you have one)*

# QUESTION 3 — MATRIX ROW SUMS

**Suggested time: 10 minutes**

## Problem

Given a matrix (a list containing rows of numbers), calculate and display the total of each row.

Use this starting matrix: matrix = [[2, 4, 6], [1, 3, 5], [10, 20, 30]].

## Examples

**Matrix above**

```
Row 1 total: 12
Row 2 total: 9
Row 3 total: 60
```

## Constraints

- Use nested loops.
- Do not use sum() or import libraries.
- Your solution should work for a matrix with a different number of rows.

## What This Tests

Nested lists, nested loops, accumulators, and indexing.

Notes / concepts to review: *(Remember to write your weakness it if you have one)*

# QUESTION 4 — BALANCED PARENTHESES

**Suggested time: 10 minutes**

## Problem

Determine whether a string contains balanced parentheses. Every opening parenthesis '(' must have a matching closing parenthesis ')' in the correct order.

You only need to handle round parentheses; ignore other character types.

## Examples

**Input: (a+b) * (c-d)**

```
Output: Balanced
```

**Input: (a+b))**

```
Output: Not balanced
```

**Input: ((a+b)**

```
Output: Not balanced
```

## Constraints

- Use a loop and a counter.
- Do not import libraries.
- If the counter ever becomes negative, the string is not balanced. At the end, the counter must be zero.

## What This Tests

Strings, loops, counters, conditionals, and algorithmic reasoning.

Notes / concepts to review: *(Remember to write your weakness it if you have one)*

# QUESTION 5 — TO-DO LIST MINI PROJECT

**Suggested time: 10 minutes**

## Problem

Build a small command-line to-do list. The user should be able to add a task, view all tasks, remove a task, or quit the program.

Display a menu repeatedly until the user chooses to quit. Number tasks when displaying them so the user can select which task to remove.

## Examples

**Menu choices: add, view, remove, quit**

```
Example flow: add “Study Python” → view tasks → remove task 1 → quit
```

## Constraints

- Store tasks in a list.
- Use a loop and if / elif / else to handle menu choices.
- Handle an empty list when viewing or removing tasks.
- Do not import libraries.

## What This Tests

Lists, loops, input, conditionals, indexing, and combining concepts in a mini project.

Notes / concepts to review: *(Remember to write your weakness it if you have one)*

## SUBMISSION

When time is up, submit your ten Python files or paste your solutions as instructed.

Remember:

- Working solution > perfect code.
- If you get stuck, write down which concept you are missing.
- Test your code rather than assuming it works.
- Do not copy solutions from the internet or use AI to generate the code during the timed attempt, unless your instructor permits it.